

Lab 2 - Python for AIML

The following topics will be covered in this lab.

1. Introduction to AIML environment with iPython Notebook.
2. Datatypes and functions in numpy.
3. Reading CSV file using numpy.
4. Datatypes and functions in pandas.
5. Reading CSV file using pandas.
6. Number generation in Python.
7. Plotting with Matplotlib library.

Instruction to complete the laboratory exercises.

1. Open python notebook file under Lab folder.
2. Read the problem statement in the exercise and its expected output.
3. Uncomment and remove the lines and fill in your answer.
4. Run your code to produce expected output.

Noted: Data files are stored in dataset folder

About iPython Notebooks

iPython Notebooks are interactive coding environments embedded in a webpage. You will be using iPython notebooks in this class. After writing your code, you can run the cell by either pressing "SHIFT"+"ENTER" or by clicking on "Run Cell" (denoted by a play symbol) in the upper bar of the notebook.

Exercise: Set test to "Hello World" in the cell below to print "Hello World" and run the two cells below.

```
In [1]: test = "Hello world"
```

```
In [2]: print ("test: " + test)
```

test: Hello world

Expected output test: Hello World

What you need to remember:

- Run your cells using SHIFT+ENTER (or "Run cell")

Datatype in Python (Dictionary)

Basic sequence datatypes such as list/array, tuple store the elements which can be of any Python datatype, including other lists and tuples. For dictionary datatype holds data in the form of key/value pair structure. The contents of a dict can be written as a series of key:value pairs within braces {}, e.g. dict = {key1:value1, key2:value2, ... }. The "empty dict" is just an empty pair of curly braces {}.

Strings, numbers, and tuples work as keys, whereas value can be of any datatype. Looking up a value which is not in the dict throws a KeyError.

Example:

- dict = {'a': 'alpha', 'o': 'omega', 'g': 'gamma'}
- print(dict)

```
In [3]: ## dict[key] = value-for-that-key

dict = {'a': 'alpha', 'o': 'omega', 'g': 'gamma'}

print(dict)
print(dict['a'])
```

```
{'a': 'alpha', 'o': 'omega', 'g': 'gamma'}
alpha
```

A dictionary iterates over its keys by default. The keys will appear in an arbitrary order. The methods dict.keys() and dict.values() return lists of the keys or values explicitly. There's also an items() which returns a list of (key, value) tuples, which is the most efficient way to examine all the key value data in the dictionary. All of these lists can be passed to the sorted() function.

```
In [4]: print(dict.values())

# accessing each key/value
for key in sorted(dict.keys()):
    print(key, dict[key])
```

```
dict_values(['alpha', 'omega', 'gamma'])
a alpha
g gamma
o omega
```

Datatypes and functions in Numpy

Numpy: Numpy is the most basic and powerful package for scientific computing and data manipulation in Python. Matrix is created using two dimension array. It is based on one main object: ndarray (which stands for N-dimensional array). Data type is specified by another NumPy object called dtype (data-type) and each nd-array is associated with only one type of dtype. In NumPy dimensions are called axes. For example, two dimensional array has 2 axes. The axis 0 is represented as row and the axis 1 as the column. Number of the dimensions and items in an array is defined by its shape. It can be displayed by calling properties of the nd-array eg. ndarray.ndim, ndarray.shape, ndarray.size

```
In [1]: # to use numpy we need to import the library
import numpy as np

# two dimensions with two axes -> 1,3
A = np.array([[1, 2, 3]])

# two dimensions with two axes -> 3,3
B = np.array([[0, 1, 2],
              [3, 4, 5],
              [6, 7, 8]])

# three dimensions with three axes -> 2,2,3
C = np.array([
    [[256, 25, 155], [211, 12, 210]],
    [[0, 0, 12], [145, 12, 100]],
])

print("A, B and C information")
print(A.ndim, B.ndim, C.ndim)      #ndim (number of dimension)
```

```
print(A.shape, B.shape, C.shape)  #shape (size of each dimension)
print(A.size, B.size, C.size)    #size (the total size of the array)
print(A)
print(B)
print(C)
```

A, B and C information

```
2 2 3
(1, 3) (3, 3) (2, 2, 3)
3 9 (2, 2, 3)
[[1 2 3]]
[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[[256 25 155]
  [211 12 210]]

 [[ 0  0 12]
  [145 12 100]]]
```

Slicing of n-d array

Slicing the array is to extract the data based on its column and row indexes and produce a new array. Numpy array slices return views rather than copies of the array data as in standard Python list operation. This default behavior is useful as it means that when we deal with huge datasets, we can access and process these datasets without the need to keep creating copy in the data buffer.

Inserting additional row or column

You can insert the additional row or column using insert function. It will insert values along the given axis (0 for row and 1 for column) before the given indices. If axis is None then the array is flattened first.

```
In [7]: # Initialize array, MA for use
MA = np.array([[1, 2, 3]])
print(MA)

# Slice the first two rows and first three columns
DD = MA[:2,:3]
print(DD)

# Slice column index 1 and print
print(MA[:,1])

# Slice the first three rows and every other column and print
print(MA[:3,::2])

# Reverse all elements and print
print(MA[::-1,::-1])

# Insert one row(axis = 0) of values 1 to the array, MA at index 1
rMA = np.insert(MA, 1, values = [1], axis = 0)
print(rMA)

# Insert one column(axis = 1) of value 1 to the array, MA at index 3
cMA = np.insert(MA, 3, values = [1], axis = 1)
print(cMA)
```

```
[[1 2 3]]
[[1 2 3]]
[2]
[[1 3]]
[[3 2 1]]
[[1 2 3]]
[1 1 1]]
[[1 2 3 1]]
```

Loading the Data using Numpy

To start a ML project, you need to load the data such as text, images, audio from various sources such as file, database, online server. In this subject, text and images files will be used as main data sources. Textual data to develop machine language model are usually stored as CSV (comma-separated values).

CSV is a simple file format stores tabular data (number and text) in plain text. In Python, CSV files can be opened and loaded using common libraries packages in Numpy and Pandas library.

Load CSV with Numpy

One approach to load CSV data file is using NumPy libraries and `numpy.genfromtxt()` function. The following is an example of loading CSV data file with the help of it.

- <https://numpy.org/doc/stable/reference/arrays.dtypes.html>
- <https://numpy.org/doc/stable/reference/generated/numpy.genfromtxt.html>

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Example: Loading the dataset above, numbers.csv file using `numpy.genfromtxt()`

```
In [7]: #1) Load the number data from numbers.csv
#2) Explore the datatype and dimension (shape)

import numpy as np

path = "./dataset/numbers.csv"
data = np.genfromtxt(path, dtype=None, names=None, delimiter=",", encoding=None)

print("shape of data : ", data.shape)
```

```
print("datatype of data : " , data.dtype)
print("sample of 3 rows of data : ", data[:3])
```

```
shape of data : (5, 5)
datatype of data : int32
sample of 3 rows of data : [[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]]
```

Datatype can be define for a specific columns while reading csv file using `genfromtxt` function. In the following example, `iris_w_header.csv` files has four columns of integer (whole number) data and string in last column. Array-protocol type strings is used to define the data type of the column. The first character specifies the kind of data and the remaining characters specify the number of bytes per item, except for Unicode, where it is interpreted as the number of characters. eg. `i4` for 32 bits signed integer (whole number) or `f8` for 64-bit floating-point number.

You can declare datatypes of column as string array as follow and use as `dtype` parameter while reading the csv file in `read_csv` function.

```
types = ['f8', 'f8', 'f8', 'f8', 'U50']
```

	sepal_length	sepal_width	petal_length	petal_width	class
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5.0	3.6	1.4	0.2	Iris-setosa
6	5.4	3.9	1.7	0.4	Iris-setosa
7	4.6	3.4	1.4	0.3	Iris-setosa
8	5.0	3.4	1.5	0.2	Iris-setosa
9	4.4	2.9	1.4	0.2	Iris-setosa
10	4.9	3.1	1.5	0.1	Iris-setosa
11	5.4	3.7	1.5	0.2	Iris-setosa
12	4.8	3.4	1.6	0.2	Iris-setosa
13	4.8	3.0	1.4	0.1	Iris-setosa
14	4.3	3.0	1.1	0.1	Iris-setosa

Example: Loading the dataset above, `Iris.csv` file using `NumPy.genfromtxt()`

```
In [3]: #3) Load the data from iris_w_header.csv
#4) Explore the datatype and dimension (shape)

import numpy as np

path = "./dataset/iris_w_header.csv"
types = ['f8', 'f8', 'f8', 'f8', 'U50']
data = np.genfromtxt(path, dtype=types, names=True, delimiter=",", encoding=None)

print("shape of data : " , data.shape)
print("datatype of data : " , data.dtype)
print("names of data : ", data.dtype.names)
print("sample of 3 rows of data : ", data[:3])
```

```

shape of data : (150,)
datatype of data : [('sepal_length', '<f8'), ('sepal_width', '<f8'), ('petal_length', '<f8'), ('petal_width', '<f8'), ('class', '<U50')]
names of data : ('sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class')
sample of 3 rows of data : [(5.1, 3.5, 1.4, 0.2, 'Iris-setosa') (4.9, 3. , 1.4, 0.2, 'Iris-setosa') (4.7, 3.2, 1.3, 0.2, 'Iris-setosa')]

```

Loading specific columns of source file

To retrieve the data from the specific columns of file using `genfromtxt()` function, you can pass the `usecols` parameters with the list of column indexes. eg. `usecols = (0,1,2,3)`

```

In [4]: data = np.genfromtxt(path, dtype=None, skip_header=1, delimiter=",", usecols = (0,1,2,3), encoding=None)
print("shape of data : ", data.shape)
print("datatype of data : ", data.dtype)
print("names of data : ", data.dtype.names)
print("sample of 3 rows of data : ", data[:3])

```

```

shape of data : (150, 4)
datatype of data : float64
names of data : None
sample of 3 rows of data : [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]]

```

Notice that the data imported is now a 2D array(matrix) because the datatypes are all same(float64).

Exercise 1:

Load the data from `numbers_ex.csv` file using numpy libraries and `numpy.genfromtxt()` function.

1. Display the shape of data, type of data.

Expected output:

- shape of data : (4, 4)
- datatype of data : float64

2. Display area (column 2) and price (column 4) data by slicing the data.

Expected output:

- [70 60 50 120]
- [910 1000 890 800]

3. Calculate the cost (cost = price / area) and insert/append cost in existing data

Expected output:

- [[1.00000000e+00 7.00000000e+01 8.00000000e+00 9.10500000e+02 1.30071429e+01]
- [2.00000000e+00 6.00000000e+01 1.30000000e+01 1.00025000e+03 1.66708333e+01]
- [3.00000000e+00 5.00000000e+01 1.80000000e+01 8.90500000e+02 1.78100000e+01]
- [4.00000000e+00 1.20000000e+02 2.30000000e+01 8.00000000e+02 6.66666667e+00]]

```

In [5]: import numpy as np

```

```

#1) Load the number data from numbers_ex.csv

# _____

# _____

#2) Explore the datatype and dimension (shape)

# _____

# _____

print()

# 3) Display area (column 2) and price (column 4) data by slicing the data.

# _____

# _____

#4) Insert the cost data (cost = price / area)

# cost = _____

print()

#4) insert/append cost to the data by column wise insert

#new_data = np.insert(data, 4, values=cost, axis=1)
#print(new_data)

```

Datatypes and functions in Pandas

Pandas is an open source, BSD-licensed library providing high-performance, easy-to-use data structures and data analysis tools for the Python programming language.

<https://pandas.pydata.org/>

Two important datatypes such as **Series** and **DataFrame** exist in Pandas.

Pandas series

Pandas series wrap both a sequence of values and a sequence of indices, which can be access with the value and index attributes in one dimension.

Pandas DataFrame

DataFrame is an analog of a two-dimensional array with both flexible row indices and flexible column names. Just as you might think of a two-dimensional array as an ordered sequence of aligned one-dimensional columns, you

can think of a DataFrame as a sequence of aligned Series objects. Here, by “aligned” we mean that they share the same index.

```
In [2]: # 1) Importing the numpy and pandas packages
# 2) Create series data using pandas

import numpy as np
import pandas as pd

data=pd.Series([0.25,0.5,0.75,1.0])
print(data)
print(data.values) #access the values of a pandas series
print(data.index)  #access the index of a pandas series
print(data[2])     #access individual value
print(data[1:3])   #access subset of a series

0    0.25
1    0.50
2    0.75
3    1.00
dtype: float64
[0.25 0.5  0.75 1.  ]
RangeIndex(start=0, stop=4, step=1)
0.75
1    0.50
2    0.75
dtype: float64
```

Pandas Dataframes

DataFrame can contain data that is:

- Pandas DataFrame
- Pandas Series: a one-dimensional labeled array capable of holding any data type with axis labels or index. An example of a Series object is one column from a DataFrame.
- Numpy ndarray, which can be a record or structured
- Two-dimensional ndarray
- Dictionaries of one-dimensional ndarray's, lists, dictionaries or Series.

```
In [3]: # 1) Creating pandas dataframe using two-dimensional np.array

my_2darray = np.array([[1, 2, 3], [4, 5, 6]])

print(pd.DataFrame(my_2darray))
```

```
0  1  2
0  1  2  3
1  4  5  6
```

```
In [4]: # 2) Creating panda dataframe using two-dimensional np.array with column

my_2darray1 = pd.DataFrame(data=my_2darray, index=range(0,2), columns=['A', 'B', 'C'])
print(my_2darray1)
```

```
   A  B  C
0  1  2  3
1  4  5  6
```

Traversing and accessing data in DataFrame

Column-wise data

Columns in DataFrame can be access using the index value of the columns property or using by label (name) of the column. It can be appended the column data into the existing DataFrame

Row-wise data using slicing and iloc

Row in DataFrame can be access using slicing method and iloc function `df[1:3]` data in row 1 to 2 `df.iloc[:]` data in all rows

Iterate Over a Pandas DataFrame

DataFrame like other data structure has iterrows method to access the cell value in DataFrame. You can iterate over the rows of your DataFrame with the help of a for loop in combination with an iterrows() call on your DataFrame:

```
In [6]: #Declare n-d array of 3x3
npdata = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

df = pd.DataFrame(data=npdata, columns=['A', 'B', 'C'])

print(df)                # print df array in Pandas DataFrame format
print(df['A'])            # print column 'A'
print(df.columns[1])      # print column 1 header

df['D'] = [14, 15, 16]    # append column 'D' to df
df['E'] = df['A'] * df['D'] # compute new data as column 'E' and append to df

print(df)                # print updated df

##Accessing the entire row data
print(df[1:2])            # access entire row 1 and before row 2
print(df.iloc[[1,2]])     # access entire row 1 & 2

##Accessing the specific cell data
print(df.iloc[0,4])       # access element at index row 0 and column 4
print(df.at[2,'E'])       # access element at index row 2 and column 'E'
```

```
   A  B  C
0  1  2  3
1  4  5  6
2  7  8  9
0    1
1    4
2    7
Name: A, dtype: int32
B
   A  B  C  D  E
0  1  2  3 14 14
1  4  5  6 15 60
2  7  8  9 16 112
   A  B  C  D  E
1  4  5  6 15 60
   A  B  C  D  E
1  4  5  6 15 60
2  7  8  9 16 112
14
112
```

Dropping unwanted columns

To remove the unwanted columns from dataframe, use drop function. The drop() function can pass inplace parameter as True and the axis parameter as 1. This tells Pandas to make direct changes/drop of the specified column(s) in the data object.

Example:

- columns_to_drop = ["C"]
- df.drop(columns = columns_to_drop, inplace = True, axis = 1)

```
In [7]: columns_to_drop = ["C"]
df.drop(columns = columns_to_drop, inplace = True, axis = 1)

print(df)
```

	A	B	D	E
0	1	2	14	14
1	4	5	15	60
2	7	8	16	112

Load CSV with Pandas

Another approach to load CSV data file is using pandas.read_csv() function that returns a pandas.DataFrame.

Pandas's read_csv function can read csv files which contains different datatypes. Statistics of each column can be displayed using describe() function of the dataframe.

eg.

- path = "./dataset/iris_w_header.csv"
- df = pd.read_csv(path)

```
In [10]: # 1) Load data from iris_w_header.csv file with Pandas.read_csv()

import pandas as pd

path = "./dataset/iris_w_header.csv"
df = pd.read_csv(path)
print("shape of data : ", df.shape)
print("\n datatype of data : " , df.dtypes)
print("\n sample of 3 rows of data : ", df[:3])
print("\n sample of 3 rows of data using head(): ", df.head(3)) # or you can using the head() function
print("\n", df.describe(), "\n") # statistic information of data usually used for Data Science Application
```

```
shape of data : (150, 5)
```

```
datatype of data : sepal_length    float64
sepal_width        float64
petal_length       float64
petal_width        float64
class              object
dtype: object
```

```
sample of 3 rows of data :      sepal_length  sepal_width  petal_length  petal_width      class
0          5.1          3.5          1.4          0.2  Iris-setosa
1          4.9          3.0          1.4          0.2  Iris-setosa
2          4.7          3.2          1.3          0.2  Iris-setosa
```

```
sample of 3 rows of data using head():      sepal_length  sepal_width  petal_length  petal_width      class
0          5.1          3.5          1.4          0.2  Iris-setosa
1          4.9          3.0          1.4          0.2  Iris-setosa
2          4.7          3.2          1.3          0.2  Iris-setosa
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Random Number Generators

Machine learning algorithm sometime requires to generate random number such as weight of the features. Sample data as random numbers sometime needs to generate when studying a machine learning model or behavior of a program. Similar to generating the random number creating numerical sequences is also useful tools in python. Following functions are frequently used in generation random or sequence of numbers.

1. random.rand()
2. arange()
3. linspace()

Numpy has a random module that generates random number.

numpy.random.rand() function creates an array of a given shape and populate it with random samples from a uniform distribution.

Example:

```
numpy.random.rand(2, 3)
```

- [[0.02161571 0.84027497 0.56929447]
- [0.12055862 0.77247655 0.79532843]]

numpy.arange() function generate sequence of numbers in ndarray type. For integer arguments the function is equivalent to the Python built-in range function, but returns an ndarray rather than a list.

Example:

```
numpy.arange(3.0)
```

- [0., 1., 2.]

numpy.linspace() function generate evenly spaced numbers over a specified interval.

Example:

numpy.linspace(0, 10, 5, endpoint=True) generates evenly spaced 5 data points from 0 to 10.

- [0. 2.5 5. 7.5 10.]

```
In [1]: import numpy as np

a = np.random.randint(100)
print(a)

b = np.random.randint(100, size=(3, 5))
print(b)

c = np.random.rand(2, 3)
print(c)
```

```
19
[[85 26 43 17 96]
 [ 8 40 68 31 46]
 [18 52 17 51 98]]
[[0.84229276 0.70442861 0.24444869]
 [0.2145043  0.27294171 0.17783976]]
```

```
In [2]: import numpy as np

r1 = np.arange(3.0)
r2 = np.arange(1, 13).reshape(3,4)
print(r1)
print(r2)

xlin = np.linspace(0, 10, 5, endpoint=True)
print(xlin)
```

```
[0. 1. 2.]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
[ 0.   2.5  5.   7.5 10. ]
```

Plotting with Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. We will use the pyplot module in Matplotlib for visualization. For the plot function, you always needs to provide x, y pair of arguments with other optional arguments.

Example:

- x = np.arange(0., 5., 0.2)
- plt.plot(x, x**2, label='quadratic')

<https://matplotlib.org/tutorials/introductory/pyplot.html>

```
In [3]: import numpy as np
import matplotlib.pyplot as plt

# evenly sampled time at 200ms intervals
x = np.arange(0., 5., 0.2)
```

```

print(x)

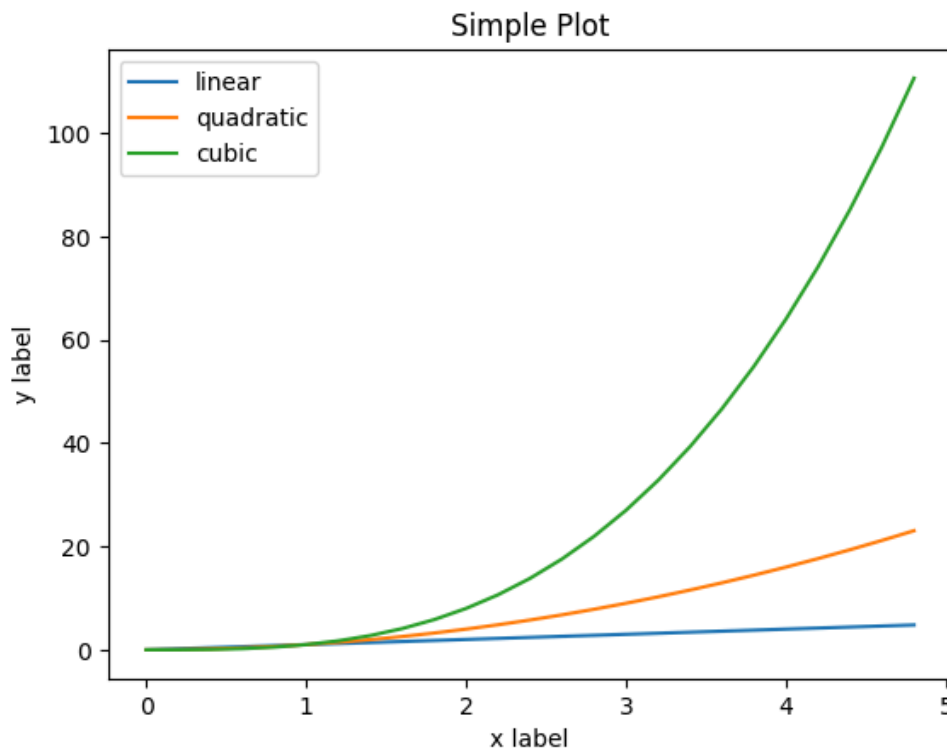
plt.plot(x, x, label='linear') # Plot some data on the (implicit) axes.
plt.plot(x, x**2, label='quadratic') # etc.
plt.plot(x, x**3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend()
plt.savefig('.\images\simple_plot.png')
plt.show()

```

```

[0.  0.2 0.4 0.6 0.8 1.  1.2 1.4 1.6 1.8 2.  2.2 2.4 2.6 2.8 3.  3.2 3.4
 3.6 3.8 4.  4.2 4.4 4.6 4.8]

```



Matplotlib Character Description

- '-' solid line style
- '--' dashed line style
- '-.' dash-dot line style
- ':' dotted line style
- '.' point marker
- ',' pixel marker
- 'o' circle marker
- 'v' triangle_down marker
- '^' triangle_up marker
- '<' triangle_left marker
- '>' triangle_right marker
- '1' tri_down marker
- '2' tri_up marker
- '3' tri_left marker
- '4' tri_right marker
- 's' square marker

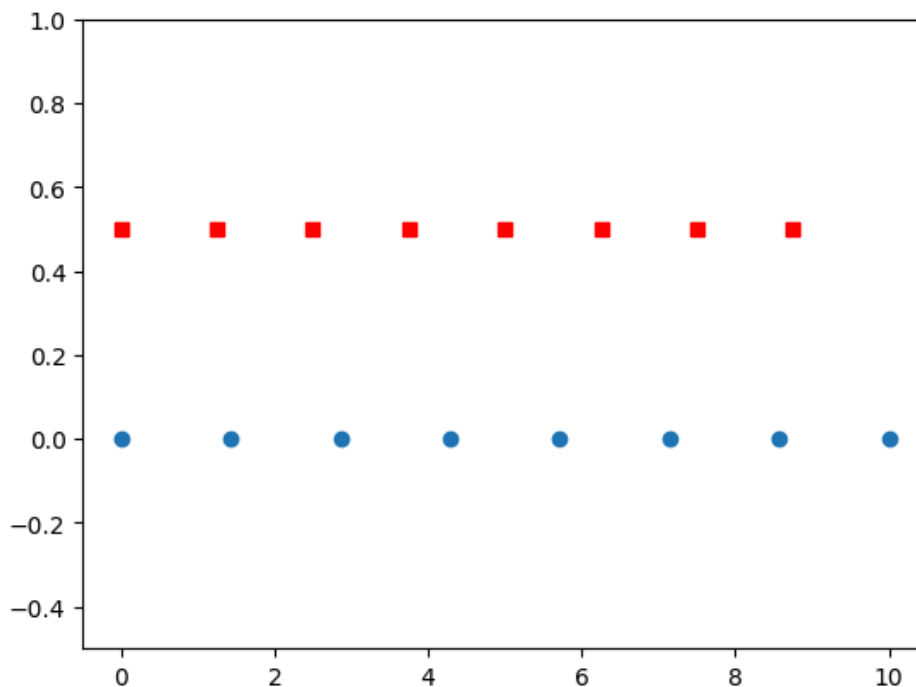
- 'p' pentagon marker
- '*' star marker
- 'h' hexagon1 marker
- 'H' hexagon2 marker
- '+' plus marker
- 'x' x marker
- 'D' diamond marker
- 'd' thin_diamond marker
- '|' vline marker
- '_' hline marker

Color Indicators

- 'b' blue
- 'g' green
- 'r' red
- 'c' cyan
- 'm' magenta
- 'y' yellow
- 'k' black
- 'w' white

```
In [4]: import numpy as np
import matplotlib.pyplot as plt

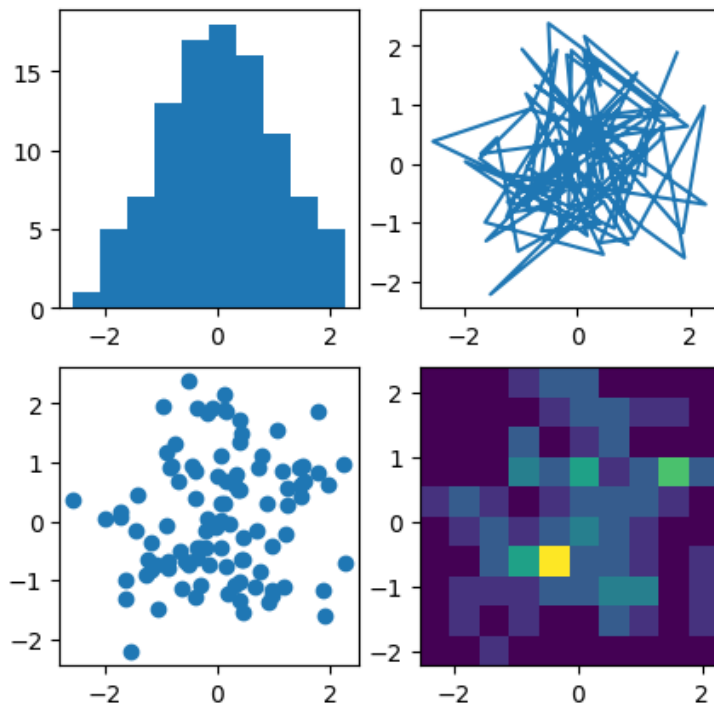
y = np.zeros(8)
x1 = np.linspace(0, 10, 8, endpoint=True)
x2 = np.linspace(0, 10, 8, endpoint=False)
plt.plot(x1, y, 'o', x2, y + 0.5, 'sr') # first plot is represented by circle and second plot is r
plt.ylim([-0.5, 1]) #(-0.5, 1)
plt.show()
```



Plotting a figure with multiple subplots

```
In [5]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
data = np.random.randn(2, 100)  # 2 x 100 array of normal-distributed random data
fig, axs = plt.subplots(2, 2, figsize=(5, 5))
axs[0, 0].hist(data[0])
axs[1, 0].scatter(data[0], data[1])
axs[0, 1].plot(data[0], data[1])
axs[1, 1].hist2d(data[0], data[1])
plt.show()
```



Exercise 2

Load the data from iris_with_header.csv using pandas library and display the dimension of the data, statistics summary and top 3 rows of the data. Plot the sepal_length and sepal_width data as scatter chart as shown in expected output.

Expected Output:

1. Load and display the data from iris_w_header.csv

shape of data : (150, 5)

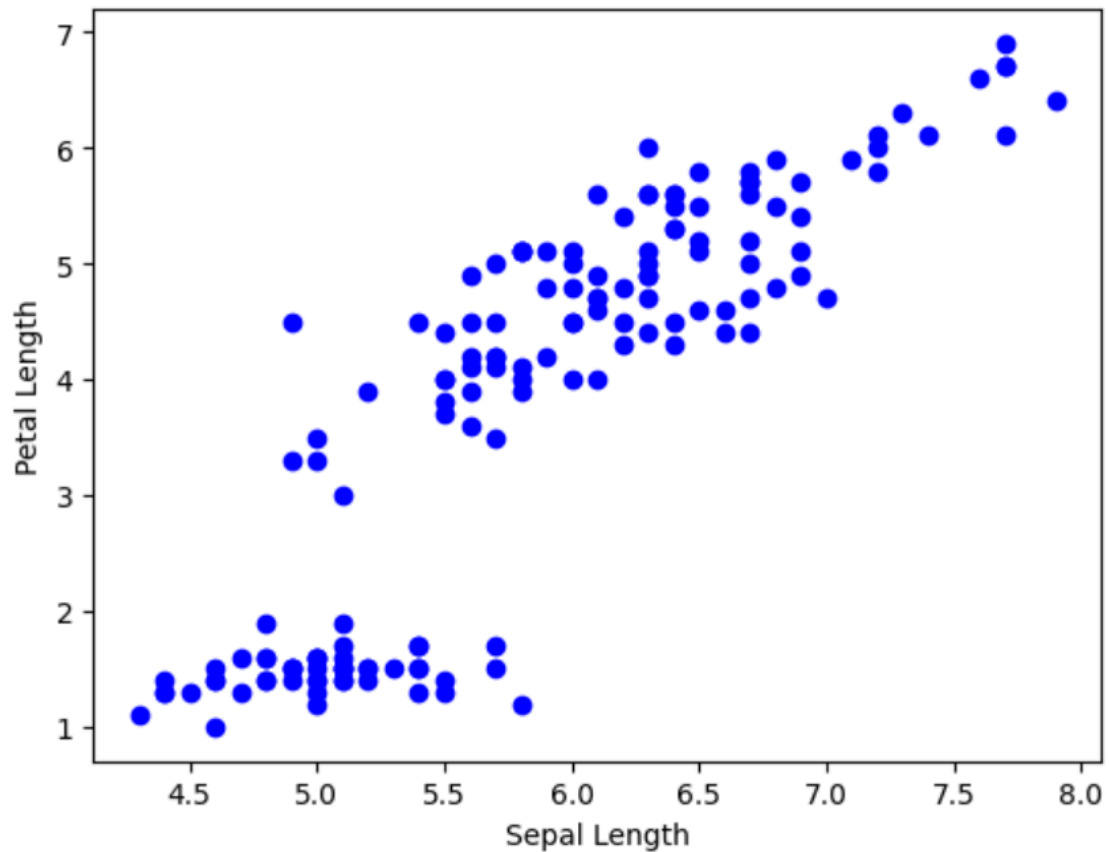
	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161

	sepal_length	sepal_width	petal_length	petal_width
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

sample of 3 rows of data using head():

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa

2. Scatter plot using petal_length vs sepal_length



```
In [6]: import pandas as pd
import matplotlib.pyplot as plt

# 1) Load data and display dimension, statistics summary and top 3 rows

# Load the data

#

#
```



```
# display dimension
```

```
# _____
```

```
# display statistics summary
```

```
# _____
```

```
# display top 3 rows
```

```
# _____
```

```
In [8]: # 2) Scatter plot using petal_length verse sepal_length
```

```
# _____
```

```
# _____
```

```
# _____
```

```
#plt.show()
```

```
In [ ]:
```